Mentimeter

Racket: Introducere

Mentimeter

PP

- programare procedurala: C
- programare orientata obiect: Java
- programare functionala: Racket, Haskell
- programare logica: Prolog

Mentimeter

Efecte laterale

- prezente in programarea procedurala/orientata obiect
- nu intoarcem numai un rezultat, modificam si starea unor entitati - variabile, structuri, etc

Mentimeter

Programare functionala = fara efecte laterale

- programare atemporala
- fara atribuiri, nu exista secvente de comenzi, etc
- **programele = compuneri + aplicari de functii**

Mentimeter

Asemanare cu axiomele unui TDA

→ TDA = tip de date abstract (de la AA)

Mentimeter

Exemplu 0

- sum lista: calculeaza suma elementelor din lista
- sum [] = 0
- sum (x:xs) = x + (sum xs)

Mentimeter

Exemplu 1

- insert element list: adauga elementul in lista pastrand lista sortata crescator
- insert y [] = y
- insert y (x:xs) = if y < x then y:(x:xs) if else x:(insert y xs)

Mentimeter

Exemplu 2

- insertion_sort lista: sorteaza lista folosind algoritmul insertion sort
- insertion_sort [] = []
- insertion_sort (x:xs) = insert x (insertion_sort xs)

Mentimeter

Wishful thinking

- putem folosi alte functii inainte sa le definim
- ex: pot folosi insert la insertion_sort si sa o definesc dupa

Mentimeter

Racket 0

- derivat din Lisp
- tipat **dinamic** - nu verificam tipul valorilor la compilare
- tipat **strong** - nu se face conversie automata la un anumit tip

Mentimeter

Racket 1

- cu **evaluare aplicativa** = argumentele functiilor sunt evaluate inainte de aplicarea functiei
- multiparadigma (suporta si unele componente imperative)

Mentimeter

Tipuri de evaluari

- aplicativa (evaluam argumentele inainte de aplicarea functiei)
- normala (evaluam argumentele odata ce avem nevoie de ele)

Mentimeter

Exemplu evaluare aplicativa (Racket)

- (define (first l) (car l))
- (first (append '(1 2) '(3 4))) = (first '(1 2 3 4)) = (car '(1 2 3 4)) = 1

Mentimeter

Exemplu evaluare normala (Haskell)

- first l = (head l)
- first ([1, 2] ++ [3, 4]) = head ([1, 2] ++ [3, 4]) = head [1, 2, 3, 4] = 1

Mentimeter

Apeluri functii vs liste

- paranteza deschisa => urmeaza un apel de functie
- daca vrem o lista => ' inainte de paranteza

Mentimeter

Exemplu: (= 1 2)

- = este o **functie**
- aplicata pe cei doi parametri: 1, 2
- intoarce false pentru 1 != 2

Mentimeter

Tipuri de date simple

- true(#t), false(#f)
- numar(ex: 1, 2, etc)
- simbol(ex: 'a, 'abc, etc)

Mentimeter

Tipuri de date compuse

- pereche(ex: '(1 2))
- lista(ex: '(1 2 3 4))
- string(ex: "ceva")
- vectori

Mentimeter

Simbol (literali): 'simbol

- 1+ caractere fara spatii
- ' evita evaluarea simbolului
- daca lipseste ' => se cauta variabila cu numele simbolului

Mentimeter

Perechi

Mentimeter

cons a b

- creeaza o pereche cu a first si b second (. intre ele, altfel e lista)
- (cons 1 2) => '(1 . 2)
- (cons '(1) 2) => '((1) . 2)
- daar, (cons '(1) '(2)) => '((1) 2) // lista, nu pereche

Mentimeter

car (primul element din pereche)

- (car '()) => eroare
- (car '(1 . 2)) => 1
- (car '((1) 2)) => '(1)

Mentimeter

cdr (al doilea element din pereche)

- (cdr '()) => eroare
- (cdr '(1 . 2)) => 2
- (cdr '(1 . (2))) => '(2)

Mentimeter

Liste

- pot sa aiba elemente cu tip diferit: (list 1 'a 5 100)
- perechi generalizate (**pereche dintre primul element si restul listei**)

Mentimeter

cons a b = adauga elementul a in lista b

- (cons 1 null) => '(1)
- (cons 1 '(2 3)) => '(1 2 3)
- (cons '(1 2) '(3)) => '((1 2) 3)

Mentimeter

cons a b - liste vs perechi

- daca b e element => rezulta o pereche
- daca b e lista => rezulta o lista

Mentimeter

list ... = creeaza o lista din argumente

- (list null) => '(())
- (list 1 2 (+ 2 4)) => '(1 2 6) // mai intai se evalueaza (+ 2 4) dupa se face lista
- daar, '(1 2 (+ 2 4)) => '(1 2 (+ 2 4)) // **daca e apostrof in fata nu se mai evalueaza dupa**
- apostrof = un fel de escape

Mentimeter

car, cdr, etc 0

- (car '(1 2 3)) => 1
- (cdr '(1 2 3)) => '(2 3)
- (cdr '(1 2)) => '(2) // la liste cdr intoarce **mereu** o lista

Mentimeter

car, cdr, etc 1

- (car (cdr '(1 2 3))) => 2
- (cdr (car (cdr '(1 (2 3))))) => '(3)
- (cdadr '(1 (2 3))) => '(3) // ultimele 2 sunt echivalente

Mentimeter

take/drop n l = intoarce/scoate primele n elemente din l

- (take '(1 2 3) 2) => '(1 2)
- (take '(1 2 3) 20) // eroare (trebuie cel putin 20 elemente)
- (drop '(1 2 3) 2) => '(3)
- (drop '(1 2 3) 20) // eroare

Mentimeter

take-right/drop-right n l

- asemanator cu take/drop, dar elementele se extrag acum din dreapta
- (take-right '(1 2 3) 2) => '(2 3)
- (drop-right '(1 2 3) 2) => '(1)
- (take-right '(1 2 3) 20), (drop-right '(1 2 3) 20) // eroare

Mentimeter

Alte functii pe liste 0

- (append l1 l2 l3 ...) poate primi **oricate** liste
- (null? l)
- (length l)

Mentimeter

Alte functii pe liste 1

- (reverse l)
- (list? l)

Mentimeter

define

- folosit pentru definirea variabilelor, functiilor, etc
- **nu se poate defini acelasi simbol de mai multe ori**

Mentimeter

Exemple define 0

- (define a 1) // nu intoarce nimic
- a => 1 // cauta identificatorul cu numele a si il afiseaza
- (define b a) // evalueaza a si salveaza b = 1
- b => 1

Mentimeter

Exemple define 1

- (define c 'a) // **apostrof** => a **nu se evalueaza**
- c => 'a
- (define d (+ 1 (+ (max 1 10) (min -8 10)))) // se evalueaza pe rand expresiile
- d => 3

Mentimeter

Functii lambda

- (lambda (lista argumente) valoare intoarsa)
- ex: (lambda (x y) (max x y))
- ex aplicare functie lambda: ((lambda (x y) (max x y)) 1 10)

Mentimeter

Functii lambda + define

- (define maxi (lambda (x y) (max x y)))
- (maxi (maxi 1 80) -10) // 80
- (define comp (lambda (f g x)(f (g x))))
- (comp list list 1) // '((1))

Mentimeter

Functii utile

- aritmetice: +, -, *, /, modulo, quotient
- relationali: <, <=, >, >=, =, eq?, equal?
- logici: not, and, or, zero?, positive?, negative?, even?, odd?

Mentimeter

Atentie la erori 0

- (or #t (> (/ 1 0) 2)) // #t
- orice valoare are (> (/ 1 0) 2), #t sau aia tot #t da
- **nu se mai evalueaza (> (/ 1 0) 2) => nu se genereaza eroare**
- la fel, (and #f (> (/ 1 0) 2)) intoarce mereu #f si nu se va genera eroare

Mentimeter

Atentie la erori 1

- daar (and #t (> (/ 1 0) 2)) si (or #f (> (/ 1 0) 2)) genereaza erori
- la ambele trebuie evaluata paranteza si acolo se genereaza si eroare

Mentimeter

eq? vs equal?

- eq? a b: verifica daca a si b refera acelasi obiect
- equal? a b: verifica daca a si b au aceeasi valoare

Mentimeter

Exemple eq?, equal?

- (eq? (list 1 2) (list 1 2)) // #f
- (eq? '(1 2) '(1 2)) // #f pentru ca sunt 2 liste diferite chiar daca s-a folosit '()
- (equal? (list 1 2) (list 1 2)) // #t

Mentimeter

(if testexpr return_true return_false) - exemple

- (if (> a 0) a (* -1 a))
- (if (= b 0) (if (> a 0) a (* -1 a)) (+ a (+ a 1)))

Mentimeter

Alt exemplu if

- (if (= 1 2) (/ 1 0) (+ 1 2)) ;; 3
- mai intai se evalueaza conditia si in functie de valoarea ei(#t/#f), **se evalueaza numai una dintre ramuri(then/else)**

Mentimeter

(cond (testexpr1 exp1) .. (else exp...)) - exemplu

- (cond
- ((= a 1) "1")
- ((= a 10) "10")
- (else "?"))

Mentimeter

Alt exemplu cond

- (define a 1)
- (cond
- ((< a 10) "digit") // se face match aici si se afiseaza "digit"
- ((> a 0) "positive")
- (else "no idea"))

Mentimeter

cond = un fel de switch

- se opreste la prima conditie adevarata si evalueaza rezultatul intors
- **restul cazurilor nu se mai verifica**

Mentimeter

Comentarii

→ cu ; in fata

Mentimeter

Exemple lab

- factorial
- suma elementelor dintr-o lista